PROJECT PLAN · DRAFT FOR YOUR REVIEW · NOT YET APPROVED

fintech-ab-platform

Plan v1.0: a $0 A/B testing platform for a fictional Colombian fintech

Synthetic events from payments, lending and savings → Kafka → Databricks Lakehouse (Bronze / Silver / 41 Gold materialized views) → statistically valid experiment readouts with a ship / don't-ship decision.

Date	2026-10-15
Prepared for	Pipe199x (github.com/Pipe199x)
Prepared by	Claude Code (Claude Opus 5.5), acting as technical lead
Status	Draft for review. Nothing gets built until you approve. Then Phase 0 starts only when you say so.
Evidence	Every free-tier and platform claim cites an official page, checked on 2026-10-15.

How to read this plan. Sections 1–9 hold the nine contents you asked for: (1) executive summary and scope · (2) decisions log · (3) architecture: Path A, Path B and the Phase 0 decision rule · (4) zero-cost bill of materials · (5) phases with acceptance criteria, verification and effort · (6) risk register · (7) draft catalog of the 41 Gold views · (8) pre-registration drafts · (9) assumptions and open questions. *Context* and *Section 0* explain the starting point and what leaving plan mode authorizes.

Contents

Context . . . 3
0. Leaving plan mode: render the PDF for your review, then STOP . . . 4
1. Executive summary and scope . . . 5
1.1 Summary . . . 5
1.2 In scope . . . 5
1.3 Out of scope . . . 5
1.4 Additions and deviations from the brief . . . 5
2. Decisions log . . . 7
2.1 Interview: every question asked and your answer (AskUserQuestion, 2026-10-15) . . . 7
2.2 Verified instead of asked . . . 7
2.3 Lead decisions (my calls; override them at approval) . . . 7
3. Architecture . . . 9
3.1 Path A (direct). Used only if the Phase 0 probes pass. . . . 9
3.2 Path B (guaranteed $0). The default unless Phase 0 proves A. . . . 10
3.3 lakehouse-bridge commit protocol: why at-least-once plus idempotent names gives zero Bronze duplicates . . . 11
3.4 Phase 0 decision rule (Path A vs B) . . . 12
3.5 Topics, keys, contract . . . 13
3.6 Datasets by layer (one pipeline) . . . 13
3.7 Offsets and consumer groups . . . 14
3.8 Ingestion options (justified) . . . 14
3.9 Silver design . . . 15
3.10 Gold factory . . . 16
3.11 Analysis method (notebook `10_ab_analysis`, library `fintech_ab.stats`) . . . 16
3.12 Plain Structured Streaming track (payments only) . . . 16
3.13 Observability, performance, Delta . . . 17
3.14 Orchestration and CI/CD . . . 17
3.15 Compliance by design (Ley 1581 de 2012; Ley 1266 de 2008, amended by Ley 2157 de 2021) . . . 17
3.16 Sizing and throughput (power analysis → population → volume) . . . 18
4. Zero-cost bill of materials (all sources checked 2026-10-15; page update date in brackets) . . . 19
5. Phases . . . 21
Phase 0: Feasibility spike (6–8 h) . . . 21
Phase 1: Producer + contracts (12–16 h) . . . 22
Phase 2: Consumer groups + Bronze (10–14 h) . . . 23
Phase 3: Silver (12–16 h) . . . 23
Phase 4: Gold, 41 MVs (12–16 h) . . . 24
Phase 5: Analysis + validation (12–16 h) . . . 24
Phase 6: Orchestration + CI/CD + SSS track (10–14 h) . . . 24
Phase 7: Performance + observability + Delta (6–10 h) . . . 25
Phase 8: Documentation (6–10 h) . . . 25
6. Risk register . . . 26
7. Gold catalog: draft of the 41 materialized views . . . 27
8. Pre-registration drafts . . . 30
9. Assumptions and open questions . . . 31
Appendix A. Repo tree (target) . . . 32
Appendix B. Deliverables → phase . . . 33
Verification (end-to-end) . . . 34

A portfolio-grade A/B testing platform for a fictional Colombian fintech, built at $0. Synthetic events from 3 lines of business flow through Kafka into a Databricks Lakehouse (Bronze, Silver, Gold on Delta). Gold has 41 materialized views. The output is statistically valid experiment readouts, each ending in a ship / don't-ship decision.

Context

Why. You asked for an end-to-end experimentation platform that a fintech data team would recognize as production-grade. The constraints are a hard $0 budget, Databricks Free Edition, the Aiven Kafka free tier, and your Windows + WSL2 laptop. This plan is the approval gate. Nothing gets built until you approve it.

Starting point (verified 2026-10-15, read-only):

- `D:\abtesting` is empty and not a git repo.
- **Windows:** Python 3.13.2, Node 22.14, Git 2.47 (user `Pipe199x`), `gh` 2.93 logged in as **Pipe199x** (scopes: repo, gist, read:org). Power BI Desktop **2.148.878.0** is installed. Docker Desktop is not.
- **WSL2:** Ubuntu **26.04 LTS** (kernel 6.6, systemd on), Python **3.14.4 with no pip/venv/ensurepip**, OpenJDK **25.0.3**, curl, tar, xz, jq and git. No Docker, pandoc, typst or node. Chromium runtime libraries are present. 7.7 GiB RAM, 8 vCPU, 948 GB free. `sudo` needs your password.
- No `~/.databrickscfg` and no Aiven CLI config exist on either side.

Intended outcome: an approved plan. Then `docs/plan/PLAN.md` + `docs/plan/PLAN.pdf` are rendered offline, and we STOP. After that come Phases 0–8, each ending in a stop point where I wait for your results.

0. Leaving plan mode: render the PDF for your review, then STOP

Plan mode blocks every file write and install. The PDF can only be produced after plan mode ends. **Accepting the exit from plan mode authorizes only steps 1–5 below.** It does not approve the plan.

You approve the plan **after reading the PDF**. Either you ask for changes (I revise PLAN.md and re-render), or you say "start Phase 0". Until then: no `git init`, no GitHub repo, no Aiven or Databricks actions, no Phase 0.

1. **Write the docs.** `docs/plan/PLAN.md` is this approved plan verbatim, except that the Mermaid blocks become `![](diagrams/*.png)` with the sources kept beside them. The diagrams go in `docs/plan/diagrams/{path_a,path_b,bridge_protocol,phase0_rule}.mmd`.
2. **Install the docs toolchain** in WSL2 user space, with no sudo. Each archive downloads into its own new empty temp directory and is checked against its GitHub asset digest or the Node.js `SHASUMS256.txt`:
   - pandoc **3.12.1** `linux-amd64.tar.gz` → `~/.local/opt`, symlinked into `~/.local/bin` (GPL-2.0-or-later).
   - Typst **0.15.1** `x86_64-unknown-linux-musl.tar.xz` → `~/.local/opt` (Apache-2.0).
   - Node.js **24.22.0 LTS** tarball → `~/.local/opt`. Then `npm install --prefix ~/.local/opt/mmdc @mermaid-js/mermaid-cli@12.0.0` (MIT). Puppeteer fetches Chrome headless shell once. After that, rendering runs fully offline.
3. **Render.**
   - Diagrams: `mmdc -i X.mmd -o X.png -s 3 -b white -p puppeteer.json`. The config passes `--no-sandbox`, because Ubuntu 26.04 restricts unprivileged user namespaces; this is acceptable for our own trusted diagrams. Output is PNG, not SVG: Typst's SVG renderer drops Mermaid's `foreignObject` labels.
   - Document: `pandoc PLAN.md -f markdown -t typst -s -o build/PLAN.typ` (A4; wide tables on landscape pages via raw `{=typst}` blocks), then `typst compile build/PLAN.typ PLAN.pdf`.
   - Fallback if Chrome won't start in WSL: run the same pinned mmdc through Windows Node 22. Also offline and no admin rights.
4. **Check.**
   - `typst compile build/PLAN.typ 'build/preview/p-{0p}.png' --ppi 50`. The PNG count is the page count.
   - I open the preview pages that carry diagrams and confirm they're visible.
   - Print `wslpath -w /mnt/d/abtesting/docs/plan/PLAN.pdf`, which should be `D:\abtesting\docs\plan\PLAN.pdf`.
5. **STOP.** No `git init`, no Phase 0, until you say so.

How commands run during the project: I execute WSL commands with `wsl.exe -d Ubuntu -- bash -lc '…'` in `/mnt/d/abtesting`. Anything interactive (`sudo`, `databricks auth login`, consoles, Power BI) I hand to you as exact lines.

1. Executive summary and scope

1.1 Summary

A simulator built on Faker `es_CO` and confluent-kafka plays 30,000 synthetic users of a Colombian fintech over 92 simulated days:

- P2P/QR/card **payments**
- digital **microcredit**
- **savings pockets**

Three concurrent experiments run on top: EXP-PAY-001, EXP-LEND-002 and EXP-SAV-003. The simulator injects **known true effects** and scheduled faults.

Events go to 5 Kafka topics (2 partitions each), identical locally (Docker, KRaft) and on Aiven's free tier. They reach Databricks Free Edition by one of two paths:

- **Path A:** Spark's Kafka source, used only if Phase 0 proves serverless can reach and authenticate to Aiven.
- **Path B:** `lakehouse-bridge`, a real consumer group in WSL2 that lands idempotently named files in a Unity Catalog volume for Auto Loader. It is the guaranteed-$0 path, and it is built either way.

One triggered Lakeflow Spark Declarative Pipeline builds everything:

- Bronze: raw streaming tables.
- Silver: typed, deduplicated and quarantined streaming tables, plus an SCD2 users table with hashed and masked PII.
- Gold: 41 metadata-driven materialized views.

An analysis notebook then:

1. checks validity (SRM, A/A, invariants);
2. runs pre-registered tests (z-test, Welch/bootstrap, CUPED, Holm/BH, fixed horizon);
3. writes `gold.experiment_results` and one-page memos;
4. **proves itself** against the injected ground truth.

A Lakeflow Job orchestrates the runs, Declarative Automation Bundles deploy them, and GitHub Actions on a public repo runs CI. Power BI Desktop and AI/BI dashboards consume Gold.

1.2 In scope

Brief items 1–10 and deliverables 0–6, as written:

- the producer and its contracts;
- both consumer groups;
- Bronze, Silver, and Gold (41 views);
- the analysis, with its correctness proofs;
- the plain Structured Streaming comparison track;
- the performance and observability lab;
- orchestration, tests and CI/CD;
- compliance-by-design;
- ADRs, the runbook and the README.

1.3 Out of scope

Item	Why
Real customers and real PII	You required synthetic data only. Free Edition also reserves the right to train on workspace data.
Power BI Service sharing	Needs Pro ($14/user/month). We ship a `.pbit` template plus screenshots instead.
Confluent Cloud, Databricks trial or paid workspaces, larger GitHub runners	Rejected by the zero-cost policy.
Sequential testing (mSPRT)	You chose fixed horizon (Q6). There is no peeking. The cumulative-lift view is labeled "monitoring only".
Avro + Schema Registry	ADR-004 explains the call. Karapace exists on Aiven's free tier, but we use JSON with JSON Schema contracts in Git.
Kafka Connect, MirrorMaker 2	Not offered on Aiven's free tier.
SLAs, HA, multi-region, real notification delivery, ML models	Portfolio scope.

1.4 Additions and deviations from the brief

None of these is silent. Each is explained.

ID	Change	Reason
A1	Add event type `notification_opt_out` to `fintech.savings.v1`.	EXP-SAV-003's opt-out guardrail needs a data source, and the listed savings events don't include one.
A2	Add two **validation fixtures** to `fintech.experiments.v1`: `EXP-AA-000` (A/A) and `EXP-SRM-900`, which has the SRM toggle on (5% of treatment exposure logs dropped).	Lets the A/A check and SRM detection run on real pipeline data without contaminating the three business experiments. These are not business experiments and have no pre-registration beyond a note.
A3	Landing path is `topic/epoch/partition/firstOffset-lastOffset.jsonl.gz` rather than `topic/partition/first-last`.	When an idle Aiven service powers off, topic data is deleted. Offsets can then restart, and without an epoch, new files would collide with old names (§3.3).
A4	Bronze adds `source_epoch`, `_ingest_path` and `_source_file` on **both** paths.	Keeps the schema identical across paths and makes lineage auditable.
A5	`schema_version` lives on the **envelope**: every event in a message shares the producer's contract version. JSON Schemas still exist per event type, per version.	Makes "explicit schema per schema_version" parse cleanly with `from_json`.
A6	`gold.experiment_results` is a Delta **table** written by the notebook, so it is not among the 41 MVs. It adds `metric_role`, `p_value_adj`, `method`, `data_cutoff_ts`, `analysis_run_id` and `code_version`. The experiment registry is a *private* pipeline dataset.	An MV can't be written by a notebook. The extra columns are for auditability.
A7	One volume, `ops.lakehouse_files`, with folders `landing/`, `checkpoints/` and `artifacts/`.	Matches "a volume for landing files and checkpoints". The CA cert, producer manifests and memo copies live in `artifacts/`.

2. Decisions log

2.1 Interview: every question asked and your answer (AskUserQuestion, 2026-10-15)

#	Question	Options offered (recommended first)	Your answer	Consequence
Q1	Which accounts do you already have? (multi-select; GitHub verified separately)	Databricks Free Edition · LinkedIn-verified · Aiven account · Aiven free Kafka running	**Databricks Free Edition** only	Not LinkedIn-verified and no Aiven account. Phase 0 gets two STOP points: Aiven sign-up, and LinkedIn verification if Path A fails.
Q2	Hours per week?	~10 h (Rec.) · ~5 h · ~20 h	**~20 h/week**	About 103 h of effort over **about 5–6 weeks**.
Q3	Repo name (public, for free Actions)?	fintech-ab-platform (Rec.) · fintech-ab-lakehouse · colombia-fintech-experimentation	**fintech-ab-platform**	`github.com/Pipe199x/fintech-ab-platform`, created in Phase 0 after your OK. The local folder stays `D:\abtesting`.
Q4	Plan language?	English (Rec.) · Spanish · English + Spanish summary	**English**	Code, comments and docs in English.
Q5	Docker runtime?	Docker Engine in WSL2 (Rec.) · Docker Desktop	**Docker Engine in WSL2**	You run about 6 `sudo` lines once (Phase 0). Apache-2.0, no license question.
Q6	Peeking policy?	Fixed horizon only (Rec.) · Add mSPRT interim looks	**Fixed horizon only**	One readout per experiment at its pre-registered cutoff.
Q7	Injected truth story?	Mixed outcomes (Rec.) · All three ship · Mixed with a true null	**Mixed outcomes**	Expected verdicts: PAY ship, LEND don't ship (lagging guardrail), SAV ship. The analysis must reach these from the data alone.

2.2 Verified instead of asked

- **GitHub:** account `Pipe199x` via `gh auth status`. All three candidate repo names were free.
- **Power BI Desktop:** installed (2.148.878.0). That is newer than the 2.143.878.0 the Databricks connector needs for client-credentials auth.
- **WSL capabilities:** as listed in Context. The project will **not** use Python 3.14; see D2.

2.3 Lead decisions (my calls; override them at approval)

ID	Decision	Rationale
D1	The repo lives at `D:\abtesting` (`/mnt/d/abtesting` in WSL). Heavy local data lives on WSL ext4: Docker named volumes and `~/fab-data` for the Spark lab and bridge spool.	Claude Code (Windows) and Power BI need Windows paths. Microsoft recommends the Linux file system for Linux-side I/O (WSL docs).
D2	Python **3.12** via `uv` (no sudo), one `uv.lock`.	Matches serverless environment v6 (Python 3.12.3). The system 3.14 has no pip or venv.
D3	Local PySpark **4.2.0** on the existing OpenJDK 25.	Spark 4.2 supports Java 17/21/25. `pyspark.testing.assertDataFrameEqual` is available.
D4	Kafka **4.2** everywhere: `apache/kafka:4.2.2` locally, `kafka_version=4.2` on Aiven.	Aiven's free tier offers 3.9, 4.1 and 4.2. Same version locally and on Aiven means no drift.
D5	Path B is built regardless of Phase 0. Path A Bronze flows sit behind the pipeline parameter `fab.ingest_path` and only if Phase 0 passes.	B is the guaranteed path and carries the kill-mid-batch proof. Bronze is identical either way.
D6	JSON + JSON Schema (draft 2020-12) contracts in Git, validated by the producer in test mode, in CI and in Silver. No Schema Registry.	ADR-004 explains why.
D7	Primary metrics are **user-level binaries**. Ratio metrics use the delta method.	Users are the randomization unit, so this keeps observations independent.
D8	N = **30,000** users over **92 simulated days** (2026-06-22 → 2026-09-21).	Sized from the power analysis (§3.16). LEND is the binding constraint.
D9	Producer settings: `enable.idempotence=true`, `acks=all`, `compression.type=zstd`, a token bucket at **50 KiB/s on the wire**, and a `throttle_cb`. Kafka message timestamps are always **send time**.	50 KiB/s is ≤40% of the most conservative Aiven limit. Simulated timestamps would be outside the 72 h retention immediately and be deleted.
D10	Silver dedup is `withWatermark("event_ts","1 hour")` + `dropDuplicatesWithinWatermark(["event_id"])`.	That is 2× the 30-minute maximum injected lateness.
D11	PII handling: keyed SHA-256 (a pepper in the secret scope) produces document, phone and email join keys. Masked display columns sit behind UC column masks. Gold carries no PII.	Ley 1581 data minimization.
D12	Power BI uses **Import** mode. A `.pbit` template is committed, never a `.pbix` with data.	DirectQuery keeps the 2X-Small warehouse awake and burns quota.
D13	Primary: two-sided α = 0.05. Secondary metrics: **Holm**. Segment cuts: **BH** (q = 0.10, exploratory). Guardrails: one-sided α = 0.05, no adjustment.	Not adjusting guardrails keeps them maximally sensitive to harm.
D14	dev/prod are separate catalogs, `fintech_ab_dev` and `fintech_ab`, chosen by a bundle variable.	If catalog creation fails, fall back to `workspace` with schema prefixes.
D15	The job failure email is `${workspace.current_user.userName}`.	Keeps your address out of the public repo.

ID	Decision	Rationale
D16	The repo is MIT-licensed, with `.env` git-ignored and GitHub secret scanning on (free for public repos).	Portfolio default.

3. Architecture

The diagrams below are rendered from the Mermaid sources in `docs/plan/diagrams/`.

3.1 Path A (direct). Used only if the Phase 0 probes pass.

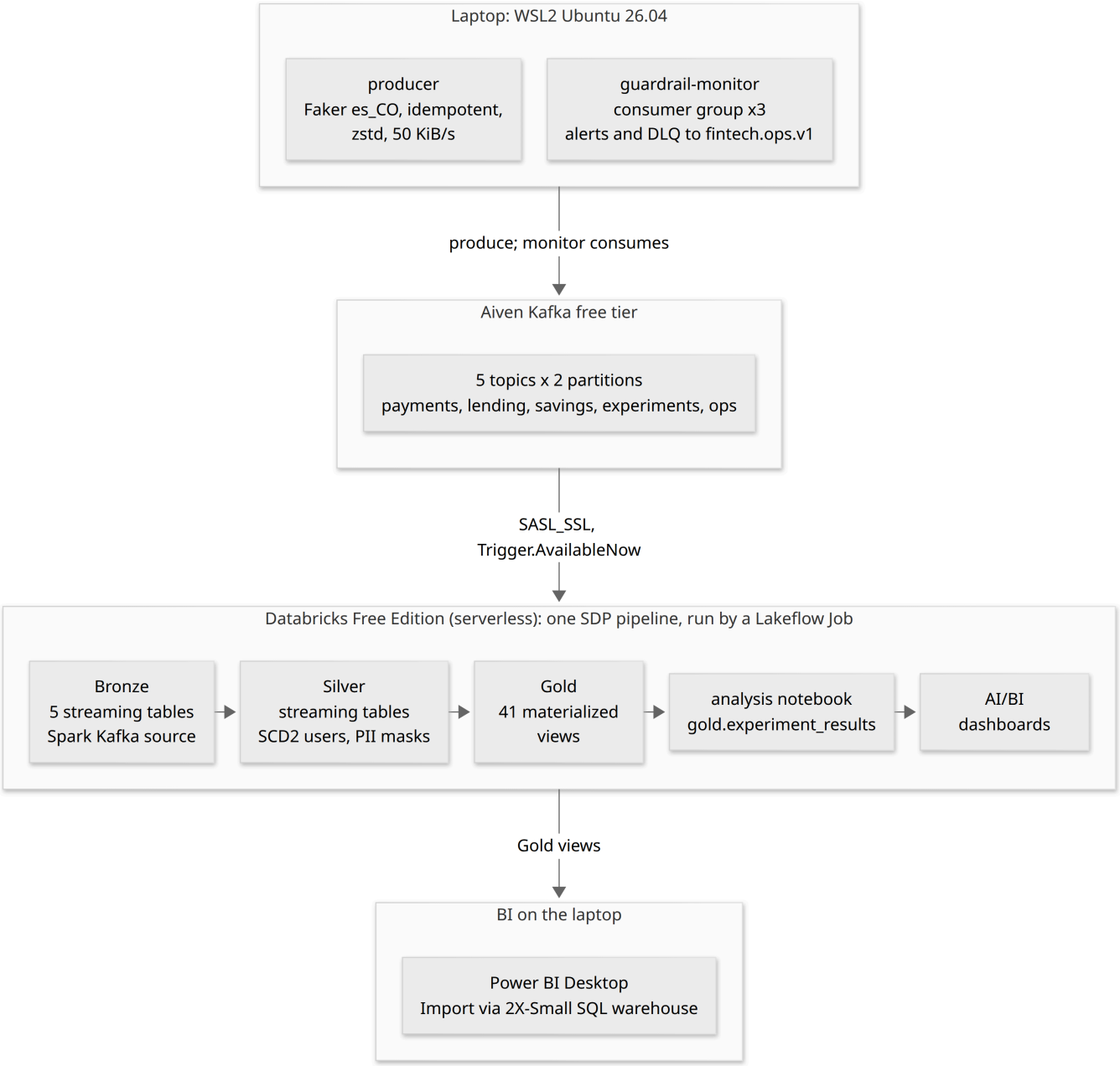


Figure 1: Path A: direct ingestion with Spark's Kafka source. Used only if the Phase 0 probes pass.

3.2 Path B (guaranteed $0). The default unless Phase 0 proves A.

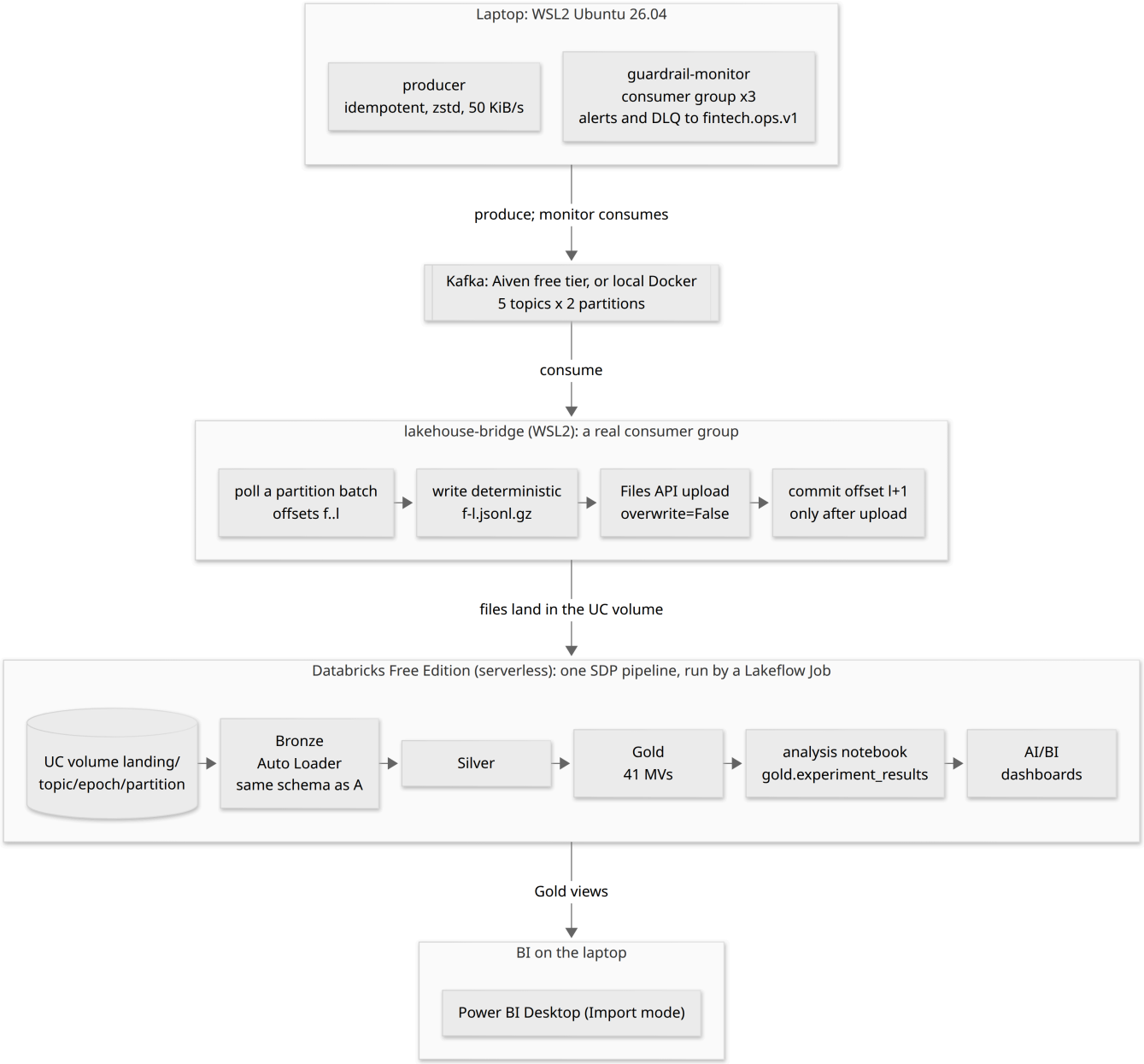


Figure 2: Path B: lakehouse-bridge plus Auto Loader. Guaranteed $0, and the default.

3.3 lakehouse-bridge commit protocol: why at-least-once plus idempotent names gives zero Bronze duplicates

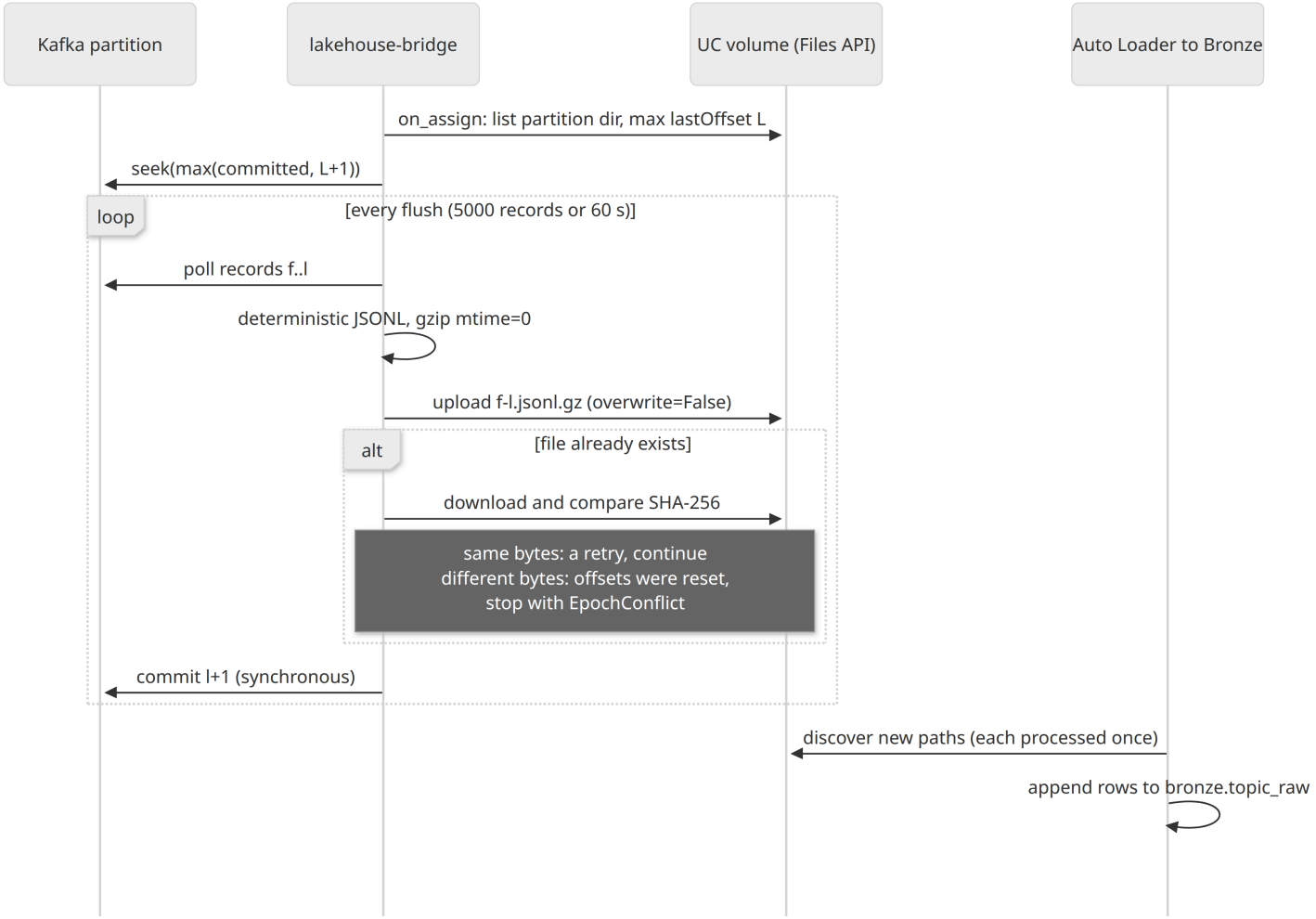


Figure 3: lakehouse-bridge commit protocol.

Argument. Each Kafka coordinate (topic, epoch, partition, offset) lives in exactly one file path, and ranges never overlap. The bridge can crash at three points:

- **Before upload:** nothing exists yet, so the batch is re-read.
- **After upload, before commit:** on restart, the reconcile step on assign seeks past the uploaded range. Without it, the next file could be `f-l'` with `l' ≠ l` and overlap the first.
- **During or after commit:** normal operation.

A re-upload of the same range produces the same path with identical bytes. Auto Loader processes each path once (Auto Loader FAQ: with `allowOverwrites=false`, "files are processed exactly once").

Proof in Phase 2. Run `--crash-after-upload 3`, which calls `os._exit` between the upload and the commit. Restart the bridge. Expect 0 duplicate coordinates and 0 offset gaps.

3.4 Phase 0 decision rule (Path A vs B)

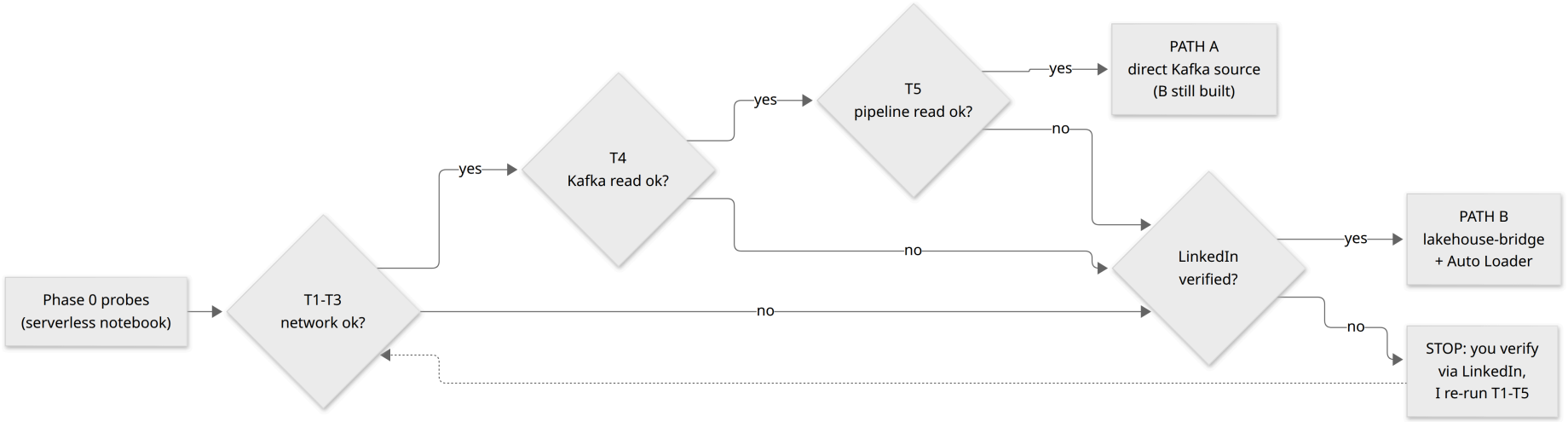


Figure 4: Phase 0 decision rule: Path A vs Path B.

Rule. Path A is chosen if and only if T1–T5 all pass from Free Edition serverless:

Probe	What it checks
T1	DNS resolution of the Aiven host
T2	TCP connect to the SASL_SSL port
T3	TLS handshake against the Aiven CA
T4	`spark.read.format("kafka")` with SCRAM-SHA-256 and a PEM truststore returns the 10 probe rows
T5	The same read inside a pipeline update

The docs **disagree** on LinkedIn verification. The AWS page (updated 2026-09-29) says verification "can raise some feature limits" and grants limited serverless GPU. Some localized mirrors also list "outbound internet access" as a post-verification benefit. So we test both before and after verification.

Evidence for each probe (pass/fail, error text) goes to `docs/evidence/phase0/` and ADR-002. **Expected outcome: Path B.** Outbound traffic is restricted to unpublished "trusted domains", and Kafka uses non-HTTP ports.

3.5 Topics, keys, contract

Topic (2 partitions; retention 72 h on both sides)	Key	Event types
`fintech.payments.v1`	user_id	payment_initiated, payment_completed, payment_failed
`fintech.lending.v1`	user_id	offer_viewed, application_submitted, approved, rejected, disbursed, installment_paid, installment_late
`fintech.savings.v1`	user_id	pocket_created, deposit, withdrawal, **notification_opt_out (A1)**
`fintech.experiments.v1`	user_id	assignment, exposure (both arms; control logs the counterfactual)
`fintech.ops.v1`	experiment_id or source topic	`kind = alert` or `kind = dlq` (flat record, no envelope)

Envelope (v1; v2 adds an optional `network_type` per event from 2026-07-20T00:00Z). One example; Phase 1 commits one example per topic under `docs/examples/`.


```
{"envelope_id":"0f6c…","schema_version":1,"producer":{"app":"fab-
simulator","version":"0.1.0","run_id":"bf-01","synthetic":true},
 "sent_at":"2026-10-20T14:03:11.204Z","session_id":"s-7f1e…",
 "user":
{"user_id":"u_0012345","city":"Medellín","department_code":"05","segment":"mass","device":"android",
         "document_number":"99xxxxxxxx","phone":"+57 399 xxx
xxxx","email":"f.salazar@example.com","push_opt_in":true},
 "events":[
  {"event_id":"e-…","event_type":"payment_initiated","event_ts":"2026-07-14T23:58:02.120Z",
   "experiments":[{"experiment_id":"EXP-PAY-001","variant":"treatment"},{"experiment_id":"EXP-
AA-000","variant":"b"}],
   "payment":
{"payment_id":"p-…","channel":"qr","amount_cop":"125000.00","currency":"COP","merchant_category":"grocery",
              "line_items":
[{"line_no":1,"item_category":"groceries","quantity":2,"unit_price_cop":"45000.00"},
{"line_no":2,"item_category":"beverages","quantity":1,"unit_price_cop":"35000.00"}]}},
  {"event_id":"e-…","event_type":"payment_completed","event_ts":"2026-07-14T23:58:04.870Z","experiments":
[…],
   "payment":{"payment_id":"p-…","checkout_latency_ms":2750,"risk":
{"fraud_flag":false,"score":"0.0312"}}}]}
```


Format rules:

- COP amounts are **strings** (later `try_cast` to DECIMAL(18,2)). Timestamps are ISO-8601 UTC with `Z`.
- Nested arrays: `experiments[]`, `line_items[]`, and lending `disbursed.installments[]` (the repayment schedule).
- Synthetic identifiers use reserved-looking ranges (`99…` documents, `+57 399…` phones, `example.com`/`example.org` email per RFC 2606). Phase 1 verifies these ranges are unassigned (open question O9).

Assignment. `bucket = int(sha256(f"{experiment_id}:{salt}:{user_id}").hexdigest()[:15], 16) % 100`; buckets 0–49 are control and 50–99 treatment. The Spark twin is `pmod(CAST(conv(substr(sha2(concat_ws(':', exp,salt,user_id),256),1,15),16,10) AS BIGINT),100)`. Parity is unit-tested.

Exposure triggers:

- PAY: checkout opened. AA-000 and SRM-900 share this trigger with independent salts.
- LEND: offer screen viewed.
- SAV: nudge-slot evaluation when an eligible user opens the app. Treatment gets the push; control logs `counterfactual=true`.

3.6 Datasets by layer (one pipeline)

Bronze (5 streaming tables, no parsing)

- Tables: `payments_raw`, `lending_raw`, `savings_raw`, `experiments_raw`, `ops_raw`.
- Columns: `key STRING, value STRING, topic, partition INT, offset BIGINT, kafka_timestamp TIMESTAMP, source_epoch, _ingest_path ('A'|'B'), _source_file, _ingested_at`.
- Table property `pipelines.reset.allowed=false`. A full refresh would truncate data that Kafka's short retention can no longer replay (pipeline update docs).

Silver (streaming tables)

- Event tables: `payments_events` (keeps `line_items` as an array), `lending_events`, `savings_events`, `experiment_events`.

- Child tables:
  - `payment_line_items` (event_id, line_no)
  - `loan_installments` (loan_id, installment_no, from `installments[]`)
  - `event_experiments` (event_id, experiment_id, from `experiments[]`)
- Ops and quality tables: `ops_alerts`, `ops_dlq`, `quarantine` (topic, partition, offset, raw value, `try_parse_json` VARIANT for debugging, `reason`).
- `users_scd2`: the AUTO CDC SCD2 target.

Gold

- 41 MVs (§7).
- Private dataset `experiment_registry`, read from `config/experiments.yml`.

ops schema

- `pipeline_event_log` (the published event log), `producer_manifests` (streaming table on `artifacts/_manifests/`), `dq_reconciliation`, `validation_results`.
- `simulation_truth`, read only by the validation step.

3.7 Offsets and consumer groups

- **Path A (Spark Kafka source):**
  - Offsets live in the pipeline-managed streaming checkpoint. Spark "manages which offsets are consumed internally" and "doesn't commit any offset" to Kafka.
  - Each query auto-generates a unique `group.id` (prefix `spark-kafka-source`), so it reads all partitions without interference.
  - `kafka.group.id` is appropriate only when broker ACLs demand a specific authorized group. Use it "with caution": concurrent or quickly-restarted queries sharing it read partial data.
  - `groupIdPrefix` fits prefix-based ACLs (Spark 4.2 Kafka guide; Databricks options).
  - Consequence: Kafka lag tools show nothing for Path A. Lag comes from the event log's `backlog_bytes`, or from end offsets minus Bronze `max(offset)`.
- **Path B (`lakehouse-bridge`):**
  - A real group: `enable.auto.commit=false`, `auto.offset.reset=earliest`, cooperative-sticky assignment.
  - Synchronous commit after a durable upload, plus the reconcile-on-assign step from §3.3.
- **`guardrail-monitor`:**
  - An independent group on the 3 LOB topics; the variant comes from each event's `experiments[]`. Different groups get **every** message; members within a group split partitions.
  - Range assignor (librdkafka default `range,roundrobin`): with 3 members and 2 partitions per topic, one member stays **idle**. Killing an active member triggers a rebalance, and the idle member takes over.
  - At-least-once delivery: `enable.auto.offset.store=false`, then `store_offsets()` after processing.
  - Rolling one-hour event-time windows per experiment × variant compute fraud rate, p95 latency, opt-out rate, withdrawal rate, and the matured-only late rate. Each instance covers the users of its own partitions, which is a random half.
  - Output: `kind=alert` records with deterministic `alert_id`, and `kind=dlq` records (with raw base64) to `fintech.ops.v1`. Per-partition lag (high watermark − position) is logged every 30 s, and a lag alert fires above a threshold.

3.8 Ingestion options (justified)

Path A

Option	Value	Why
`startingOffsets`	`earliest`	It only matters on the first start; after that the checkpoint wins. It captures everything still inside the ≤72 h retention.
`maxOffsetsPerTrigger`	200,000	Split proportionally across partitions, so event time advances evenly (protects the watermark). `AvailableNow` still drains the backlog in several bounded batches, so a quota stop mid-update loses at most one batch of work.
`failOnDataLoss`	`true` (default)	A quota stop longer than retention, or an Aiven power-off, can delete unread offsets. Silently skipping payments would bias experiment metrics (data missing not at random), so we fail loudly and follow the runbook: reset the flow checkpoint (not the table data), bump `epoch`, and re-seed. A pipeline run every 6 h is far inside 72 h.
Auth	`SASL_SSL`, SCRAM-SHA-256, `kafkashaded…ScramLoginModule`	Credentials come from the secret scope. The Aiven CA is a PEM in the volume (`kafka.ssl.truststore.type=PEM`; Phase 0 test T4) (Kafka auth docs).

Path B: Auto Loader

Option	Value	Why
`cloudFiles.format` + explicit schema	`json`	The bridge's own contract is fixed.
`cloudFiles.schemaEvolutionMode`	`none`	The landing schema is fixed.
`pathGlobFilter`	`*.jsonl.gz`	Ignores stray files.
`cloudFiles.includeExistingFiles`	`true`	Picks up the backfill.
`cloudFiles.allowOverwrites`	`false`	Exactly-once per path.
`cloudFiles.maxFilesPerTrigger`	1000	The whole backfill (~130 files) lands in one micro-batch, so the Silver watermark can't mark a lagging partition as "late".
`cloudFiles.partitionColumns`	`""`	No Hive-style inference.
File detection	directory listing	Small file counts. File events would need an external location.
`cloudFiles.cleanSource`	`OFF`	Landing files are our replay log beyond Kafka retention. Revisited in Phase 7.

3.9 Silver design

- **Parsing.**
  - `get_json_object(value,'$.schema_version')` routes each message: v1 parses with `schema_v1` and v2 with `schema_v2`, then `unionByName(allowMissingColumns=True)`.
  - Unknown versions go to quarantine with `unsupported_schema_version`. `from_json` returns null on malformed input, which goes to quarantine as `malformed_json`.
  - Amounts and timestamps use `try_cast` / `try_to_timestamp` (ANSI is on).
  - We choose `from_json` over VARIANT because it enforces the contract, yields typed and clusterable columns, and is cheaper to store. VARIANT can't be a clustering key or be grouped or compared (VARIANT vs JSON). VARIANT is used only for quarantine forensics. Details in ADR-004.
- **Expectations: which rule gets which action** (expectations).

Action	Rules	Why
`expect_or_fail`	`known_topic`; `no_clear_pii` (clear email/phone/document must be null after hashing)	These signal code or wiring bugs. Polluting Silver, or leaking PII, is worse than stopping. Triggered pipelines roll back only that flow.
`expect_or_drop`	`event_id_not_null`, `valid_event_ts`, `valid_event_type`, `amount_non_negative`, `user_id_not_null`	The complement is routed to `silver.quarantine` with a reason by an append flow, so nothing is lost.
`expect` (warn)	`latency_in_range` (50–60,000 ms), `not_future_ts` (≤ now + 5 min), `late_arrival_le_30m`	Plausibility signals that we monitor but keep.

- **Dedup.** The watermark is 1 h on `event_ts` (D10). The backfill-ordering risk (R10) is handled by landing the backfill in one micro-batch, plus exact reconciliation against the producer manifest.
- **SCD2 users.**
  - Setup: `dp.create_streaming_table("silver.users_scd2", schema=…MASK…)`, then `dp.create_auto_cdc_flow(…, keys=["user_id"], sequence_by=struct("observed_at","topic","offset"), stored_as_scd_type=2, track_history_column_list=["segment","city","department_code","device","push_opt_in"])` (create_auto_cdc_flow).
  - The source is a temporary view that unions the user objects from all 4 envelope topics and hashes them first.
  - `observed_at` is the session's first `event_ts`.
- **PII.**
  - `*_hash = sha2(concat(pepper, ':', normalized_value), 256)`. The pepper comes from `secret('fab', 'pii_pepper')` in SQL, or `dbutils.secrets.get` if the SQL function is not allowed in pipelines (open question O7).
  - Display columns: `email_masked` and `phone_last4`, with the mask `fintech_ab.ops.mask_pii(v) = CASE WHEN is_account_group_member('pii_readers') THEN v ELSE '***' END`, declared in the schema DDL (masks).
  - Column tags via `ALTER STREAMING TABLE … ALTER COLUMN … SET TAGS` (tags): `pii`, `ley_1581`, `ley_1266`, `purpose`.
  - Clear values never reach Silver. The Bronze schema is limited to the owner.
- **Assignment consistency.** Checked in Gold (`exp_assignment_consistency`). Expectations can't query other tables.

3.10 Gold factory

All pipeline code uses the current API, `from pyspark import pipelines as dp`, and never the legacy `import dlt` (Python ref: "`dlt` … replaced by `pyspark.pipelines`").

`config/gold_views.yml` defines all 41 views. Each entry carries: name, domain, grain, sources, owner, consumer, description, cluster_by, columns (name/type/comment), and the SQL logic.

The `pipelines/fab/gold_factory.py` loop calls `_register(spec)`, which creates one `@dp.materialized_view(name, comment, cluster_by, schema=<DDL with column COMMENTs>, table_properties)` per spec (materialized_view). Two details:

- **Late-binding.** The factory function gives each closure its own `spec`. A naive loop would make all 41 MVs run the last view's SQL, because pipelines call the functions later. A unit test registers 3 fake specs on a stub `dp` and asserts each captured function returns its own SQL.
- **SQL dialect.** Gold SQL stays OSS-Spark-compatible (no QUALIFY), so CI can analyze every view locally against empty Silver schemas.

Around the factory:

- The governance task generates `GRANT USE CATALOG/USE SCHEMA` plus a per-view `GRANT SELECT … TO bi_readers`.
- `tools/gen_data_dictionary.py` writes `docs/data_dictionary.md`, and CI fails if it is stale.
- Timestamps are converted to Bogotá only in Gold: `from_utc_timestamp(ts,'America/Bogota')`.

3.11 Analysis method (notebook `10_ab_analysis`, library `fintech_ab.stats`)

Ordering.

1. Validity gates, before any outcome is looked at:
   - SRM χ^2 goodness-of-fit against the designed split; the gate is $p < 0.001$.
   - Exposure-based population.
   - Invariants such as the LEND credit-policy check.
   - A/A on EXP-AA-000.
2. Tests:
   - Two-proportion z-test for rates (pooled-SE test; unpooled CI for the difference; relative lift CI by the delta method).
   - Welch t-test plus 10,000-sample bootstrap percentile CIs for heavy-tailed COP amounts; user-level resampling with a fixed seed.
   - Cluster bootstrap for the payment-level p95 latency.
   - CUPED: $\theta = \mathrm{Cov}(Y,X)/\mathrm{Var}(X)$, pooled, using pre-period covariates.
   - Delta method for ratio metrics.
   - Non-inferiority: the upper bound of the one-sided 95% interval must be below the margin.
   - Holm for secondaries; BH $q = 0.10$ for segments.
3. Outputs:
   - MERGE into `gold.experiment_results`.
   - Memos rendered to `docs/memos/EXP-*.md`, plus a copy in the volume.
4. **Only then** does `11_validation` load the ground truth.

Libraries. scipy 1.16.3 is preinstalled in serverless environment v6. statsmodels is **only in the ML base** there. So `fintech_ab.stats` is numpy + scipy, cross-checked in CI against statsmodels as an oracle, and the notebook `%pip installs` statsmodels only if PyPI is reachable (open question O6) (env v6).

Correctness proofs:

- **(a)** The injected truth falls inside each 95% CI. A local pytest coverage simulation over 500 seeds must give coverage in [0.93, 0.97], because any single interval misses 5% of the time.
- **(b)** 1,000 re-salted A/A splits of the EXP-PAY-001 control arm must give a false-positive rate in [3.65%, 6.35%] (the 95% binomial band), with a KS uniformity $p > 0.01$.
- **(c)** EXP-SRM-900's SRM p must be < 0.001, and the clean experiments' $p \geq 0.001$.

3.12 Plain Structured Streaming track (payments only)

Notebook `30_sss_payments`, run as a scheduled serverless job:

- **Q1:** Auto Loader on the landing folder (Path B) or Kafka (Path A) → `bronze.sss_payments_raw`. Append-only, with checkpoint `checkpoints/sss/payments_bronze`.

- **Q2:** a Delta stream from that table → `foreachBatch`. Each batch is parsed, exploded and de-duplicated, then MERGEd into `silver.sss_payments_events` on `event_id`, inserting only when not matched. Checkpoint: `checkpoints/sss/payments_silver`.
- Both queries use `trigger(availableNow=True)` (the only supported option is AvailableNow; Once is deprecated (serverless limits)).
- Replaying a batch inserts 0 rows. Dedup is global by key, with no watermark. That contrast is a talking point for ADR-003.

ADR-003 then compares the track with Declarative Pipelines on offsets, checkpoints, failure recovery, data quality, and effort (LOC and hours).

3.13 Observability, performance, Delta

- **Spark UI lab (local).** PySpark 4.2 in WSL on 20M generated payments with one skewed merchant (30% of rows). Show at `localhost:4040`: stages, shuffle, skew, and AQE (coalesce + skew join). Measure before and after for a broadcast join, a skew fix (AQE or salting), and file sizing (small files vs compacted). Numbers go in `docs/perf/spark-ui-lab.md`.
- **Databricks.** Query profile screenshots (there is no Spark UI on serverless). Event-log SQL for throughput (`flow_progress.metrics.num_output_rows`), expectations (`data_quality.expectations`), backlog and latency (event log). Auto Loader progress via `cloud_files_state(TABLE(…))`.
- **Lag.** Path A: end offsets minus Bronze `max(offset)`, plus `backlog_bytes`. Path B: the bridge's and monitor's lag logs.
- **Delta.** MVs reject OPTIMIZE/VACUUM, and pipelines maintain them automatically. So the demo runs on regular Delta tables (the SSS tables and `experiment_results`):
  - **OPTIMIZE:** compacts the small files that frequent `foreachBatch` MERGEs create.
  - **VACUUM:** keep 168 h, which bounds storage while preserving a week of time travel and streaming safety.
  - **Time travel:** reproduce the exact results version a memo cites (`VERSION AS OF`). Not available on MVs.
  - **CDF:** `table_changes()` audits how decisions changed between runs and feeds incremental consumers.

3.14 Orchestration and CI/CD

- **Bundle:** `databricks.yml` with targets `dev` (mode development) and `prod` (mode production).
  - In production mode, without a service principal, `run_as` + `permissions` must be explicit and paths can't be user-scoped (deployment modes). So prod uses `/Workspace/Shared/.bundle/…`.
  - Variables: `catalog`, `ingest_path`, `landing_root`.
  - The host comes from the env var, never committed.
  - Bundle resources: pipeline, jobs, dashboards, schemas, volume, secret_scope (resources).
  - The catalog is bootstrapped by SQL in Phase 0 (the community-reported workaround for Free Edition).
- **Job `fab_daily`:**
  - Runs daily at 06:00 America/Bogota.
  - Task order: `pipeline_update` → `governance` (grants, tags, masks; idempotent) → then `analysis` and `dq_report` in parallel. That is ≤2 concurrent tasks, under the limit of 5.
  - Failure emails at task level. `max_concurrent_runs: 1`.
- **Job `sss_payments`:** daily at 07:00.
- **The package** is built as a wheel and installed into the serverless job and pipeline environments.
- **GitHub Actions** (standard `ubuntu-latest`, free for public repos):
  - `ci.yml`: ruff lint/format, unit + stats tests (local Spark), contract tests, integration tests against an `apache/kafka:4.2.2` service container (producer → bridge with a local-filesystem writer → kill/restart → no dups), and `databricks bundle validate` using repo secrets (skipped on forks).
  - `deploy-dev.yml`: on push to `main`, runs `databricks bundle deploy -t dev` via `databricks/setup-cli@v1.21.0`.

3.15 Compliance by design (Ley 1581 de 2012; Ley 1266 de 2008, amended by Ley 2157 de 2021)

- **Minimization:** no direct identifiers in Silver or Gold, only peppered hashes and masked display values. Gold uses pseudonymous `user_id` only.
- **Purpose limitation:** `purpose` tags and table properties, the data dictionary, and BI access to Gold only.
- **Masking:** UC column masks.
- **Least privilege:** `bi_readers` gets USE CATALOG/SCHEMA and SELECT on the 41 views and results only. `pii_readers` can unmask. Bronze and Silver are owner-only.
- **Audit columns:** Kafka coordinates, `source_epoch`, `_source_file`, `_ingested_at` and `_processed_at` in Silver; `_refreshed_at` in Gold.

- **Ley 1266 financial habeas data:** lending late-payment data is tagged `ley_1266=dato_financiero_negativo`, and its documented retention is bounded ("temporalidad"; the amendment defines how long negative data may persist).
- **Synthetic flag:** every envelope carries `producer.synthetic=true`.

3.16 Sizing and throughput (power analysis → population → volume)

Experiment	Baseline	MDE (abs)	α / power	n per arm	Exposure probability	Users needed
EXP-PAY-001	85%	+2.0 pp	0.05 two-sided / 0.80	**4,724**	~0.95	~9,950
EXP-LEND-002	10%	+2.0 pp	0.05 / 0.80	**3,841**	~0.35	**~21,950 (binding)**
EXP-SAV-003	20%	+2.5 pp	0.05 / 0.80	**4,202**	~0.45	~18,700

Formula: $n = [z_{0.975}\cdot\sqrt{(2\bar{p}\bar{q})} + z_{0.8}\cdot\sqrt{(p_1q_1+p_2q_2)}]^2 / \delta^2$. statsmodels' arcsine method gives 4,722 / 3,840 / 4,201.

N = 30,000 is 21,950 × 1.35 safety for uncertainty in exposure rates. Expected exposed users per arm: about 14,250 (PAY), 5,250 (LEND), 6,750 (SAV).

Timeline (simulated, America/Bogota):

Window	Dates
Pre-period (CUPED)	06-22 → 07-05
Enrollment	**07-06 → 08-02** (28 days, full weeks)
schema v2 switch	07-20
Cutoff: PAY	08-03
Cutoff: SAV	08-10
Cutoff: LEND primary	08-17
Cutoff: LEND guardrail (maturity)	09-21 (last disbursal + 30-day due + 5-day grace)

Volume: about 1.3 M events in about 0.6 M messages. That is ~0.7 GB raw JSON, or ~0.15 GB on the wire with zstd at ~5×.

Aiven budget. The sources disagree on the limit:

- docs (2026-08-18): 250 KiB/s in + 250 KiB/s out;
- the create page: "250 KiB/s total";
- the launch changelog: 125 + 125.

We design for **125/125**. With 2 consumer groups (bridge + monitor), egress is 2× ingress. So the producer runs at **50 KiB/s on the wire**. The backfill takes **about 50–60 min**, with a ceiling of ~4 h if compression underperforms. Phase 1 measures the real ratio locally and prints the projected ETA before the cloud run. The local Docker backfill takes ~5 min.

4. Zero-cost bill of materials (all sources checked 2026-10-15; page update date in brackets)

Component	Free tier / license	Limits	When a limit is hit	Official source	Paid option rejected	Free fallback
Databricks Free Edition	$0, no payment method, "forever free", non-commercial use	Serverless only. Outbound internet limited to unpublished trusted domains. 1 SQL warehouse (2X-Small). 5 concurrent job tasks per account. 1 active pipeline per pipeline type. No account console or account APIs. No SSO/SCIM. Inactive accounts may be deleted.	Fair-usage quota exceeded: compute shut down for the rest of the day ("in extreme cases the rest of the month"). Data is kept.	limitations [2026-09-29]; trial vs FE [2026-09-11]	Free trial (credits expire after 14 days); paid workspaces	Develop and test on local PySpark; keep data small; run heavy refreshes on separate days.
Serverless compute (in FE)	Included	Only `AvailableNow`/`Once` triggers. No cache/persist/checkpoint, no RDD, no Spark UI. ANSI on. UTC.	Unsupported trigger → `INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED`	serverless limits [2026-10-07]	Classic clusters (paid)	Local Spark UI lab
Lakeflow SDP, Jobs, Unity Catalog, volumes, AI/BI dashboards	Included in FE	As above. Pipelines: 1,000 datasets per pipeline.	Second active pipeline blocked (semantics: open question O3)	pipeline limits [2026-10-09]	n/a	One pipeline; deploy one target at a time
Databricks CLI v1.21.0 + Declarative Automation Bundles	Databricks License, free to use with Databricks Services	n/a	n/a	bundles [2026-09-11]; github.com/databricks/cli (2026-10-14)	n/a	SDK/REST scripts
Aiven for Apache Kafka, free tier	$0, **no credit card**, "not a trial"	5 topics × 2 partitions. 250 KiB/s in + 250 KiB/s out per docs (125 + 125 per Dec-2025 changelog). Retention fixed, ≤72 h. Powers off after **24 h** with no produce/consume, and **topic data is deleted** on shutdown (config kept). 1 free service per organization. Limited users and ACLs. No Kafka Connect, MM2, tiered storage or integrations.	Throttled clients (delayed responses). Oldest data dropped at retention. Idle → power-off and data loss.	free tier [2026-08-18]; create [2026-09-09]; free-kafka; advanced params [2026-05-18]; changelog [2025-12-10]	Aiven paid plans; Confluent Cloud (expiring credits)	Local Docker Kafka. Path B runs end-to-end without Aiven.
Apache Kafka 4.2.2, `apache/kafka` image (KRaft)	Apache-2.0	Laptop RAM (7.7 GiB in WSL)	Out of memory → lower the heap	downloads (4.2.2 released 2026-09-29); image	n/a	n/a
Docker Engine (apt repo; Ubuntu 26.04 supported)	Apache-2.0 (Moby)	n/a	n/a	install	Docker Desktop paid tiers	Docker Desktop (free for personal use)
WSL2 + Ubuntu 26.04	Free	n/a	n/a	Existing install	n/a	n/a
uv 0.13.1 + CPython 3.12	Apache-2.0/MIT; PSF	n/a	n/a	github.com/astral-sh/uv (2026-10-14)	n/a	`python3-venv` via apt (needs sudo)
Python libraries	confluent-kafka 2.16.0 (Apache-2.0); Faker (MIT, `es_CO`); scipy (BSD-3, preinstalled 1.16.3); statsmodels (BSD-3, ML base only in env v6); pytest (MIT); databricks-sdk 0.153.0 (Apache-2.0); jsonschema, PyYAML (MIT);	PyPI reachability from serverless unconfirmed	`%pip` blocked → use preinstalled scipy and the in-repo stats module	env v6 [2026-09-15]; pyspark (4.2.0, 2026-07-14); Faker es_CO	n/a	Vendored implementations

Component	Free tier / license	Limits	When a limit is hit	Official source	Paid option rejected	Free fallback
	pyspark 4.2.0 (Apache-2.0); ruff (MIT)					
Java for local Spark	OpenJDK 25 (installed; GPLv2 + Classpath Exception)	Spark 4.2 supports Java 17/21/25	n/a	spark.apache.org/docs/4.2.0	n/a	Temurin 21 tarball (no sudo)
Power BI Desktop 2.148.878.0	Free download	Windows only. No publishing or sharing without Pro.	Can't share → ship `.pbit` + screenshots	pricing; Databricks connector [2026-10-09]	Pro $14/user/ month; PPU $24	AI/BI dashboards in the workspace
GitHub public repo + Actions (standard runners)	Free for public repos	Larger runners are "always charged". Standard per-job limits apply.	n/a	Actions billing	Larger runners	Self-hosted runner in WSL (free)
GitHub secret scanning	Free for public repos	n/a	n/a	secret scanning	Secret Protection (paid, private repos)	`detect-secrets` pre-commit hook
Docs toolchain	pandoc 3.12.1 (GPL-2.0+), Typst 0.15.1 (Apache-2.0), mermaid-cli 12.0.0 (MIT), Node 24.22.0 LTS (MIT), Chrome headless shell (BSD-3)	Offline after install; no sudo	n/a	GitHub releases API; nodejs.org/ dist	Online converters (forbidden), Prince (paid)	WeasyPrint (BSD)

Your Claude Code plan, laptop and internet are outside this rule, as you specified.

5. Phases

Protocol for every phase.

- Every phase ends with three things: exact commands for you to run, the expected output, and a verification query. Then a **STOP**: I wait for your results, even in auto mode.
- At the start of each phase, I re-verify any free-tier claim it depends on and date-stamp it in the BOM.
- `sql/verify/phaseN.sql` runs from WSL via `uv run python -m fintech_ab.tools.sql <file>` (Statement Execution API on the 2X-Small warehouse) or in the SQL editor.
- Effort is our joint working time. At 20 h/week, the total is **~103 h (range 86–120 h)**, about **5–6 weeks**:

Week	Work
W1	Phase 0 + Phase 1
W2	Phase 2 + start of Phase 3
W3	Rest of Phase 3 + Phase 4
W4	Phase 5 + start of Phase 6
W5	Rest of Phase 6 + Phase 7 + start of Phase 8
W6	Finish Phase 8; buffer

Heavy Databricks runs are spread across days to respect the daily quota.

Phase 0: Feasibility spike (6–8 h)

Steps

1. **Repo bootstrap.** `git init`, `.gitignore` (`.env`, `.venv`, `*.pbix`, `runs/`, `build/`), MIT LICENSE.
   - **STOP S0.1:** your OK before `gh repo create Pipe199x/fintech-ab-platform --public --source . --push`, and before enabling secret scanning / push protection.
2. **Local tools (no sudo).**
   - uv → `uv python install 3.12` → `uv sync`.
   - Databricks CLI: the `linux_amd64.tar.gz` → `~/.local/bin`.
   - **You run** `databricks auth login --host <workspace-url> --profile fab` (OAuth in the browser).
3. **Docker Engine: you run** the apt-repo lines from docs.docker.com for Ubuntu 26.04 + `sudo usermod -aG docker $USER`. I'll paste them exactly. Done now so Phase 1 doesn't wait.
4. **Databricks bootstrap** (`sql/bootstrap/00_catalog_schemas_volumes.sql`):
   - `SELECT current_metastore()` gives the region, which decides the Aiven region group.
   - `CREATE CATALOG IF NOT EXISTS fintech_ab` / `fintech_ab_dev`, then refresh Catalog Explorer. Fallback: catalog `workspace`.
   - Schemas: bronze, silver, gold, ops. Volume: `ops.lakehouse_files`.
5. **Platform capability probes.** Each records PASS or FAIL plus the fallback taken:
   - create groups `bi_readers` / `pii_readers` and a `GRANT`;
   - `databricks secrets create-scope fab` + put/get;
   - service principal + OAuth secret (else a PAT) for CI;
   - a probe pipeline, then a **second** pipeline (does "one active pipeline per type" mean existence or concurrency?);
   - `%pip install statsmodels` (PyPI reachable?).
6. **Files API probe.** `uv run python -m fintech_ab.tools.volume_probe upload --rows 100` uploads JSONL from WSL with databricks-sdk to `landing/_probe/`. Notebook `00_phase0_probes` reads it with Auto Loader (`availableNow`) into `ops.probe_autoloader`.
7. **STOP S0.2: before the Aiven sign-up.** It's free, needs no card, and is not a trial. With your OK:
   - you sign up and create **one Free-tier Kafka service** (version 4.2, the region group nearest to Databricks);
   - enable SASL (`kafka_authentication_methods.sasl`);
   - create the 5 topics × 2 partitions;
   - check whether internal topics (Karapace `_schemas`) count toward the limit. If they do, first disable the schema registry (we don't use it). Only if that's impossible, propose merging `fintech.ops.v1` into `fintech.experiments.v1` with a `record_type` discriminator, mirrored locally;
   - create service users for producer, bridge, monitor and databricks if the users/ACL limit allows (else use `avnadmin`, a documented compromise);
   - download the CA cert. Credentials go to `.env` and the secret scope.
8. **Kafka probe from WSL.** `uv run python -m fintech_ab.tools.kafka_probe --target aiven` sends and receives 10 SASL_SSL messages on `fintech.ops.v1`.
9. **Path A probes T1–T5** (§3.4) from serverless.

    - **STOP S0.3** if they fail: LinkedIn verification (no cost; it shares your name, email and verification status with Databricks). Then re-run.
10. **Power BI Desktop → SQL warehouse.** Server Hostname + HTTP Path; OAuth or PAT; Import `ops.probe_autoloader`; screenshot.
11. **Write ADR-002 (evidence) and ADR-005 (zero-cost)**, and update the BOM with the probe facts.

Acceptance:

- Catalog, schemas and volume exist, or the documented fallback is in place.
- 100 probe rows are in `ops.probe_autoloader`.
- 10/10 Kafka probe messages.
- The T1–T5 matrix is recorded and the path decided by the rule.
- Power BI has rendered probe data.
- Open questions O1–O6 are resolved.

Verification

```
databricks current-user me -p fab && databricks schemas list fintech_ab -p fab && databricks volumes list fintech_ab ops -p fab
uv run python -m fintech_ab.tools.kafka_probe --target aiven          # expect: sent=10 received=10
```

```
SELECT current_metastore();                                  -- aws:<region>:<uuid>
SELECT count(*) FROM fintech_ab.ops.probe_autoloader;        -- expect 100
SHOW GRANTS ON SCHEMA fintech_ab.gold;                       -- bi_readers listed (if groups work)
```

Phase 1: Producer + contracts (12–16 h)

Deliverables

- Config:
    - `config/topics.yml`;
    - `config/simulation.yml` (population, timeline, rates, fault schedule, **truth** block);
    - `config/experiments.yml` (registry + pre-registered parameters).
    - **The pre-registrations are frozen**: `docs/prereg/EXP-*.md` with git tag `prereg-v1` *before the first full backfill*.
- `contracts/` JSON Schemas:
    - the envelope, v1 and v2;
    - every event type, per version, with a `schema_version` const;
    - the ops alert/dlq record schemas.
- Code: `src/fintech_ab/{common,producer,tools}`.
    - Producer modes: backfill (simulated time) and live (`now()`).
    - Rate limiter: token bucket + `throttle_cb`.
- Faults:

Fault	Rate / timing
Duplicate envelopes	1%, re-sent after 0–120 s
Late events	2%, delayed 1–30 simulated minutes
Malformed JSON	0.5%
v2 switch	2026-07-20
SRM toggle	5% of treatment exposures dropped, EXP-SRM-900 only
Malformed burst	One window, for the DLQ demo

- `docker/compose.yml`: Kafka 4.2.2, KRaft, a named volume, 72 h retention.
- `tools/topics.py` creates and describes the topics, idempotently, on both local and Aiven.
- Each run writes a manifest: counts by topic, event type and fault, plus a hash of the truth block.
- `docs/examples/<topic>.json`: one example per topic.
- Tests:
    - assignment parity (Python vs Spark) and uniformity χ^2;
    - fault-rate tolerances;
    - every non-faulty sample validates against its schema, and every malformed one fails;
    - Faker `es_CO` fields;
    - an integration round trip on local Kafka.
- A small Aiven smoke run (500 users × 1 day). **The full Aiven backfill happens in Phase 2, when the bridge can drain it inside retention.**

Acceptance:

- `describe` shows the same 5 × 2 layout locally and on Aiven.
- Fault rates within ±20% relative of their targets.
- Treatment share within 0.5 ± 0.01.
- Producer logs show idempotence + `acks=all`.
- A local backfill of 30k users finishes, with its measured compression ratio and the projected Aiven ETA printed.

Verification

```
docker compose -f docker/compose.yml up -d && uv run python -m fintech_ab.tools.topics ensure --target
local
uv run pytest tests/unit tests/contract -q && uv run pytest tests/integration -m kafka -q
uv run python -m fintech_ab.producer backfill --target local --users 30000 --seed 20260706   # prints
manifest + Aiven ETA
```

Phase 2: Consumer groups + Bronze (10–14 h)

Deliverables

- `src/fintech_ab/bridge` with the §3.3 protocol. The writer interface has two implementations: Files API and local directory (for CI). Flags: `--crash-after-upload N` and `--epoch`.
- `src/fintech_ab/monitor` (§3.7).
- The bundle skeleton (`databricks.yml`, `resources/fab_pipeline.yml`: serverless, triggered, event log published to `ops.pipeline_event_log`).
- `pipelines/fab/bronze.py` (with the path switch). ADR-001 (streaming table vs MV for ingestion).
- **The full Aiven backfill runs with the bridge consuming in parallel.**
- A ≤15-minute **continuous-mode live demo**, then back to triggered.
- **Kill test:** `--crash-after-upload 3`, then restart.
- **Monitor demo:** 3 instances → one idle → kill one → rebalance → alerts and DLQ appear in `ops_raw`.

Acceptance:

- 5 Bronze tables with the identical schema.
- Per (topic, epoch, partition): row count = last − first + 1, with 0 duplicates and 0 gaps, including after the kill test.
- `describe` shows independent offsets for `lakehouse-bridge` and `guardrail-monitor`, plus the idle member and the rebalance.

Verification

```
databricks bundle deploy -t dev && databricks bundle run -t dev fab_pipeline
uv run python -m fintech_ab.tools.groups describe --target aiven --group lakehouse-bridge
uv run python -m fintech_ab.tools.groups describe --target aiven --group guardrail-monitor
```

```
SELECT topic, source_epoch, `partition`, `offset`, count(*) c FROM fintech_ab_dev.bronze.payments_raw GROUP
BY ALL HAVING c > 1;  -- 0 rows
SELECT topic, source_epoch, `partition`, max(`offset`) - min(`offset`) + 1 - count(DISTINCT `offset`) AS
gaps
  FROM fintech_ab_dev.bronze.payments_raw GROUP BY ALL;
-- all 0 (repeat for the other 4 Bronze tables)
```

Phase 3: Silver (12–16 h)

Deliverables

- The parse/route/explode/dedupe flows and child tables (§3.6, §3.9).
- `users_scd2`, the quarantine table, the PII mask function and tags.
- Pure transform functions in `src/fintech_ab/transforms`, tested with `assertDataFrameEqual` on local Spark. The pipeline files stay thin.
- ADR-004.

Acceptance (via `ops.dq_reconciliation`, all rows PASS):

- Silver unique events = produced − malformed − contract-invalid.
- Duplicates removed = injected **exactly**.
- All injected late events are retained.
- Quarantine counts match the injected faults by reason.
- v2 rows have `network_type`; v1 rows have null.
- SCD2: exactly 1 current row per user, and contiguous `__START_AT`/`__END_AT`.
- 0 clear-PII values in Silver/Gold. Masks show `***` to non-members. Tags are visible.

Verification

```
SELECT * FROM fintech_ab_dev.ops.dq_reconciliation WHERE status <> 'PASS';                    -- 0
rows
SELECT user_id FROM fintech_ab_dev.silver.users_scd2 GROUP BY user_id HAVING count_if(__END_AT IS NULL) <>
1; -- 0 rows
SELECT * FROM fintech_ab_dev.information_schema.column_tags WHERE schema_name = 'silver';
```

Phase 4: Gold, 41 MVs (12–16 h)

Deliverables

- `config/gold_views.yml` (41 entries), `gold_factory.py`, and the private `experiment_registry`.
- The governance task (grants, tags), `tools/gen_data_dictionary.py` → `docs/data_dictionary.md`.
- Tests: the YAML model, the closure test, local SQL analysis of all 41 views, and the dictionary staleness check.
- An AI/BI "Business" dashboard.

Acceptance:

- 41 MVs: 12 exp, 10 pay, 9 lend, 7 sav, 3 exec.
- Every MV has a table comment and column comments, non-empty `clusteringColumns`, and SELECT granted to `bi_readers`.
- Spot reconciliations pass: Gold totals = Silver totals; SRM flags EXP-SRM-900 only.

Verification

```
SELECT split_part(table_name,'_',1) dom, count(*) FROM fintech_ab_dev.information_schema.tables
 WHERE table_schema='gold' AND table_type='MATERIALIZED_VIEW' GROUP BY 1;                    -- exp 12, pay
10, lend 9, sav 7, exec 3
SELECT count(DISTINCT table_name) FROM fintech_ab_dev.information_schema.table_privileges
 WHERE table_schema='gold' AND grantee='bi_readers' AND privilege_type='SELECT';             -- >= 41
DESCRIBE DETAIL fintech_ab_dev.gold.pay_daily_kpis;                                          --
clusteringColumns [date_bogota, channel]
```

Phase 5: Analysis + validation (12–16 h)

Deliverables

- `src/fintech_ab/stats`, plus the statsmodels oracle tests and the coverage simulation.
- Notebooks `10_ab_analysis` and `11_validation`.
- `gold.experiment_results` and 3 memos.
- The AI/BI "Experimentation" dashboard and the Power BI `.pbit`.

Acceptance:

- Decisions follow the pre-registered rules mechanically.
- (a) truth inside the CI for every metric with an injected truth;
- (b) A/A FPR inside [3.65%, 6.35%];
- (c) SRM detected for EXP-SRM-900 only.
- Results are reproducible with a pinned seed and code version.

Verification

```
uv run pytest tests/stats -q
```

```
SELECT experiment_id, metric, metric_role, variant, n, estimate, abs_lift, rel_lift, ci_low, ci_high,
p_value, srm_p_value, decision
  FROM fintech_ab_dev.gold.experiment_results ORDER BY 1,3,2,4;
SELECT check_name, passed, details FROM fintech_ab_dev.ops.validation_results;                -- all
passed
```

Phase 6: Orchestration + CI/CD + SSS track (10–14 h)

Deliverables

- The `fab_daily` and `sss_payments` jobs and notifications.
- `notebooks/30_sss_payments` and ADR-003.
- `ci.yml` / `deploy-dev.yml`.
- Prod deploy. If O3 forbids two pipelines: `bundle destroy -t dev` first, documented.

Acceptance:

- An end-to-end job run is green.
- A forced failure (parameter) sends an email.
- ≤5 concurrent tasks at any time.
- The SSS replay inserts 0 rows (MERGE metrics).
- CI is green on a PR, and a push to `main` deploys dev.

Verification

```
databricks bundle validate -t prod && databricks bundle deploy -t prod && databricks bundle run -t prod
fab_daily
gh run list --limit 3
```

```
DESCRIBE HISTORY fintech_ab.silver.sss_payments_events;      -- replay run shows numTargetRowsInserted = 0
```

Phase 7: Performance + observability + Delta (6–10 h)

Deliverables

- `perf/spark_ui_lab` (local), with measured before/after numbers and screenshots.
- Query profile screenshots and the event-log SQL pack.
- Lag panels.
- `notebooks/40_delta_ops`: OPTIMIZE, VACUUM 168 h, time travel, CDF.

Acceptance: every optimization has a metric that improved (wall time, shuffle bytes, or max/median task ratio). Each Delta feature has one justified demo with output.

Verification

```
uv run python perf/spark_ui_lab/run_lab.py --rows 20000000 --scenario all   # open http://localhost:4040
```

```
SELECT * FROM fintech_ab.ops.pipeline_event_log WHERE event_type = 'flow_progress' ORDER BY timestamp DESC
LIMIT 50;
```

Phase 8: Documentation (6–10 h)

Deliverables

- The README: problem, architecture, $0 cost table, how to run, results, and screenshots of the pipeline graph, query profile, Spark UI and dashboard.
- `docs/runbook.md`: zero to first readout, exact commands. It is kept current as each phase lands.
- The final ADRs, catalog and memos.
- Re-rendered PLAN.pdf if the plan changed (changelog section).

Acceptance: a clean-clone dry run of the runbook reaches a readout, and every link resolves.

6. Risk register

ID	Risk	L	I	Mitigation	Phase
R1	Free Edition blocks Aiven: outbound traffic restricted, non-HTTP ports	H	M	Path B is the default and is built regardless. LinkedIn retest. Evidence in ADR-002.	0
R2	Daily quota stops compute mid-work	M	M	Triggered (not continuous) runs. Small data. Import-mode BI. Heavy runs split across days. Bronze has `reset.allowed=false`. Bounded batches.	All
R3	Aiven powers off after 24 h idle and **deletes topic data**	H	M	Delta Bronze plus landing files are the system of record. Bridge runs during producer sessions. Epoch paths + `overwrite=False` hash check. Re-seed runbook.	2
R4	Aiven throughput throttling (125 vs 250 KiB/s ambiguity)	M	L	50 KiB/s on the wire, zstd, `throttle_cb`, live ETA. Local backfill fallback.	1–2
R5	Internal topics (`_schemas`) consume the 5-topic quota	M	M	Disable the schema registry. Else the documented ops→experiments merge plan.	0
R6	"1 active pipeline per type" blocks dev + prod	M	M	Phase 0 probe. Else one target deployed at a time; CI deploys dev only.	0, 6
R7	Can't create a catalog, groups or a secret scope	M	M	Fallbacks: `workspace` catalog; grants to the user principal; masks keyed on `current_user()`; secrets via job parameters plus `.env` (documented weakness).	0
R8	No service principal for CI	M	L	A PAT in GitHub secrets, ≤90-day lifetime, rotation note. Never printed. Skipped on forks.	0, 6
R9	PyPI blocked, so no statsmodels on serverless	M	L	numpy + scipy `fintech_ab.stats`, with statsmodels as the CI oracle.	5
R10	Backfill ordering lets the event-time watermark drop "late" rows (partition-skewed micro-batches)	M	H	Backfill lands in one Auto Loader / Bronze micro-batch. Proportional Kafka splits on Path A. A 1 h watermark. Exact reconciliation against the producer manifest catches any drop.	3
R11	Full refresh of Bronze loses data that Kafka can't replay	M	H	`pipelines.reset.allowed=false`. Landing files kept as a replay log.	2
R12	Clear PII in Bronze raw payloads	H (by design)	M	Bronze is owner-only. Hash at Silver. `expect_or_fail(no_clear_pii)`. Gold has no PII. Synthetic flag.	3
R13	Secrets leak into the public repo	L	H	`.env` git-ignored, `detect-secrets`, GitHub secret scanning, secret scope, review before every push.	All
R14	Statistical malpractice (peeking, multiplicity, SRM)	M	H	Prereg tag before data. Fixed horizon. SRM gate. Holm/BH. A/A simulations. Truth loaded last.	1, 5
R15	Seed luck: the truth falls outside one CI by chance (5%)	M	L	Pre-registered seed, honest report, and the coverage simulation proves calibration.	5
R16	Lending maturity bias (unmatured loans counted as "not late")	M	H	Guardrail computed on matured loans only. Maturity share reported per arm. Final cutoff 09-21.	5
R17	Kafka timestamps set to simulated time → instant retention deletion	M	H	Always use send time (D9). Unit test.	1
R18	Free tier terms change mid-project	M	M	Re-verify the BOM at each phase start. Fallbacks listed.	All
R19	WSL 9P I/O slowness	M	L	Heavy data on ext4 (D1).	1, 7
R20	Free Edition deletes an inactive account	L	M	Weekly activity. Everything is rebuildable from IaC + runbook.	All
R21	Generated document numbers or phones collide with real people	M	M	Reserved-looking ranges + `example.com`. Verified in Phase 1 (O9). Hashed in Silver. No enrichment or lookups.	1
R22	Puppeteer/Chrome fails in WSL (sandbox)	M	L	`--no-sandbox` config. Windows-side mmdc fallback.	post-approval
R23	Continuous-mode demo burns quota	M	L	≤15 min, then switch back to triggered.	2

7. Gold catalog: draft of the 41 materialized views

All views live in `<catalog>.gold`, have SELECT granted to `bi_readers`, carry no PII, and use Bogotá dates. Owners by domain: experimentation-platform, payments-analytics, lending-analytics, savings-analytics, exec-analytics.

Experimentation (12)

#	View	Grain	Sources	Purpose	Consumer	cluster_by
1	exp_exposures	experiment × user	silver.experiment_events	First assignment and first exposure: variant, bucket, timestamps, exposure count. The exposure-based population.	Analysis notebook, DS	experiment_id, first_exposure_date
2	exp_srm_monitor	experiment × as_of_date	exp_exposures, registry	Cumulative exposed users per arm vs the designed split. χ^2 statistic; flag when $\chi^2 > 10.83$ (df=1, p<0.001).	Experiment owners	experiment_id, as_of_date
3	exp_assignment_consistency	experiment × variant	silver.event_experiments, exp_exposures	Mismatches between `experiments[]` tags on LOB events and the exposure log	DQ, DS	experiment_id
4	exp_user_metrics_pay	user (PAY exposed)	exp_exposures, silver.payments_events	First-checkout completion (primary), attempt counts, TPV, fraud flags, latency array	Analysis	variant, first_exposure_date
5	exp_user_metrics_lend	user (LEND exposed)	exp_exposures, silver.lending_events, silver.loan_installments	14-day funnel flags, disbursal, first-installment maturity and late flag, credit band	Analysis, risk	variant, first_exposure_date
6	exp_user_metrics_sav	user (SAV exposed)	exp_exposures, silver.savings_events	Deposit within 7 days (primary), amount, withdrawal and opt-out within 7 days	Analysis	variant, first_exposure_date
7	exp_cuped_covariates	experiment × user	exp_exposures, Silver LOB events (pre-period)	Pre-period covariates per primary metric, plus a has_pre flag	Analysis (CUPED)	experiment_id
8	exp_funnel_daily	experiment × variant × step × exposure_date	exp_user_metrics_*	Funnel counts and step conversion	Product dashboards	experiment_id, exposure_date
9	exp_guardrails_daily	experiment × variant × date × metric	exp_user_metrics_*, silver.ops_alerts	Guardrail values plus guardrail-monitor alert counts	Owners, risk	experiment_id, metric_date
10	exp_segment_cuts	experiment × variant × dimension × value	exp_user_metrics_*, users_scd2 (as of exposure)	Numerators and denominators by city, segment, device and network_type (exploratory; BH downstream)	DS	experiment_id, dimension
11	exp_cumulative_lift_daily	experiment × as_of_date × variant	exp_user_metrics_*	Cumulative rate and naive lift. **Monitoring only.**	Dashboard (warning)	experiment_id, as_of_date
12	exp_sample_size_progress	experiment	exp_exposures, registry	Required vs exposed n per arm, % complete, projected date	Owners, exec	experiment_id

Payments (10)

#	View	Grain	Sources	Purpose	Consumer	cluster_by
13	pay_daily_kpis	date × channel	silver.payments_events	Initiated, completed, failed, completion rate, TPV, average ticket, payers	Payments PM, exec	date_bogota, channel
14	pay_hourly_volume	date × hour × channel	payments_events	Hourly volume heatmap	Ops, capacity	date_bogota, channel
15	pay_failure_reasons	date × channel × reason	payments_events	Failures and their share	Payments eng	date_bogota, failure_reason
16	pay_latency_percentiles	date × channel × device	payments_events	p50/p95/p99 checkout latency	SRE	date_bogota, channel

#	View	Grain	Sources	Purpose	Consumer	cluster_by
17	pay_fraud_daily	date × channel × department	payments_events	Fraud-flag rate, flagged TPV	Risk	date_bogota, department_code
18	pay_merchant_category_daily	date × merchant_category	payments_events	TPV, transactions, ticket size	Commercial	date_bogota, merchant_category
19	pay_line_item_category_daily	date × item_category	payment_line_items, payments_events	Items, quantity, amount	Analytics	date_bogota, item_category
20	pay_city_daily	date × department × city	payments_events	TPV, payers, completion by city	Regional managers	date_bogota, department_code
21	pay_ticket_size_buckets	date × channel × bucket	payments_events	Ticket-size distribution in COP	Pricing	date_bogota, channel
22	pay_user_activity_daily	user × date	payments_events	Per-user daily payments and TPV (pseudonymous id)	CRM analytics	date_bogota, user_id

Lending (9)

#	View	Grain	Sources	Purpose	Consumer	cluster_by
23	lend_offer_funnel_daily	date × product × segment	silver.lending_events	View → submit → approve → disburse	Lending PM	date_bogota, product
24	lend_approval_by_band	date × credit_band	lending_events	Approval rate per band (credit-policy monitor)	Credit risk	date_bogota, credit_band
25	lend_rejection_reasons	date × reason	lending_events	Rejections and their share	Credit risk	date_bogota
26	lend_disbursements_daily	date × product × term	lending_events	Loans, disbursed COP, average EA rate	Finance	date_bogota, product
27	lend_loan_book	loan	lending_events, loan_installments	Principal, outstanding balance, status	Finance, risk	status, disbursed_date
28	lend_installment_schedule	loan × installment	loan_installments, lending_events	Due date, amount, paid timestamp, days late	Collections	due_date
29	lend_first_installment_vintage	disbursement_week × product	lend_installment_schedule	Matured share; late rate on matured loans	Risk (vintages)	disbursement_week
30	lend_delinquency_buckets_daily	as_of_date × dpd_bucket	lend_installment_schedule	Loans and outstanding by days-past-due bucket	Collections	as_of_date
31	lend_time_to_disburse	application_date × product	lending_events	p50/p95 minutes from application to disbursal	Lending ops	application_date

Savings (7)

#	View	Grain	Sources	Purpose	Consumer	cluster_by
32	sav_pockets_daily	date × goal_type	silver.savings_events	Pockets created, creators	Savings PM	date_bogota, goal_type
33	sav_flows_daily	date × channel	savings_events	Deposits, withdrawals, net flow in COP	Treasury	date_bogota
34	sav_pocket_balances	pocket	savings_events	Current balance, last activity, goal	Product, CRM	goal_type
35	sav_deposit_7d_cohorts	pocket_cohort_week × segment	savings_events, users_scd2	Share of new pockets with a deposit within 7 days (baseline)	Growth	cohort_week
36	sav_withdrawal_rates_weekly	week × segment	savings_events	Withdrawal rate and average withdrawal	Savings PM, risk	week_start
37	sav_notification_optouts_daily	date × channel	savings_events	Opt-outs by push/email/SMS	CRM, compliance	date_bogota
38	sav_saver_segments	segment × department	sav_pocket_balances, users_scd2	Active savers, average balance, penetration	Marketing	segment

Executive (3)

#	View	Grain	Sources	Purpose	Consumer	cluster_by
39	exec_daily_overview	date	Silver LOB events, pay/lend/sav Gold	Daily active users, TPV, disbursed amount, net savings flow	Leadership	date_bogota
40	exec_cross_lob_engagement	week × segment	Silver LOB events	Users active in 1/2/3 lines of business; cross-sell	Strategy	week_start
41	exec_experiment_portfolio	experiment	exp_sample_size_progress, exp_srm_monitor, exp_guardrails_daily	Status, days running, n per arm, SRM flag, alerts	Leadership; decisions dashboard	experiment_id

8. Pre-registration drafts

These are frozen as `docs/prereg/*.md` with tag `prereg-v1` before the full backfill. The simulator's truth block is deliberately **not** referenced here.

Common rules:

- Randomization unit: `user_id`, by deterministic salted hash, 50/50.
- Exposure-based analysis: only exposed users count, and metric windows start at first exposure.
- SRM gate: χ^2 $p < 0.001$ means the readout is invalid.
- A/A sanity check via EXP-AA-000.
- Fixed horizon: one analysis after the cutoff, no interim decisions.
- Simulation seed 20260706.

Field	EXP-PAY-001 New checkout flow	EXP-LEND-002 Redesigned loan-offer screen	EXP-SAV-003 Savings nudge
Hypothesis	One-tap checkout reduces abandonment, so first-checkout completion rises without more fraud or slower p95	A clearer offer screen raises offer→disbursal conversion. **Credit policy (approval rules, rates) is identical across arms; only the UX differs.**	A personalized push nudge raises the share of users who deposit within 7 days
Trigger / exposure	Checkout opened	Loan offer screen viewed	Nudge-slot evaluation for eligible users (control logs the counterfactual)
Primary metric	Share of exposed users whose first post-exposure checkout completes within 30 min	Share of exposed users disbursed within 14 days of first view	Share of exposed users with ≥1 deposit within 7 days
Secondary (Holm)	Payment-level completion ratio (delta method); TPV per user (Welch + bootstrap, CUPED on pre-period TPV); payments per user	Application rate; disbursed COP per exposed user (bootstrap)	Deposit COP within 7 days (Welch + bootstrap, CUPED); pockets created within 7 days
Guardrails	Fraud-flag rate (baseline ~1%), NI margin +0.50 pp (one-sided 95% upper bound < margin; ~79% power at the planned n). p95 checkout latency, margin +100 ms (user-cluster bootstrap)	First-installment late rate on **matured loans only** (due + 5-day grace ≤ cutoff). Don't ship if harm is significant (one-sided α=0.05) **or** the point estimate exceeds +3.0 pp. Powered at 80% to detect ≥5.6 pp at the planned n; this limitation is disclosed.	Opt-out within 7 days, NI margin +1.0 pp. Withdrawal within 7 days, NI margin +2.0 pp.
Invariants	Pre-period checkout opens balanced	Approval rate by credit band equal across arms (χ^2 per band, Holm). Offered rate identical.	Pre-period deposits balanced
Baseline / MDE	85% / +2.0 pp	10% / +2.0 pp	20% / +2.5 pp
α / power / n per arm	0.05 two-sided / 0.80 / **4,724** (9,448 total)	0.05 / 0.80 / **3,841** (7,682)	0.05 / 0.80 / **4,202** (8,404)
Duration	Enrollment 2026-07-06 → 08-02 (4 full weeks). Cutoff 08-03 00:00 Bogotá.	Same enrollment + 14-day conversion window + 35-day maturity. **Single final readout** after cutoff 09-21.	Same enrollment + 7-day window. Cutoff 08-10.
CUPED covariate	Pre-period completion ratio	Pre-period offer views / prior loan flag	Pre-period deposit count
Decision rule	**Ship** if SRM OK, primary $p < 0.05$ with lift > 0, and both guardrails non-inferior. Else don't ship / iterate.	**Ship** if SRM OK, invariants hold, primary $p < 0.05$ with lift > 0, and the guardrail is not breached. Else don't ship.	**Ship** if SRM OK, primary $p < 0.05$ with lift > 0, and both NI tests pass.
Threats	Novelty effect (4-week window); fraud is a rare event	Maturity bias, handled by the matured-only metric and the per-arm maturity share; selection of marginal borrowers	Notification fatigue; overlap with other experiments (orthogonal salts)

9. Assumptions and open questions

Assumptions

- A1 The Free Edition workspace is on AWS. Its region is learned in Phase 0 via `current_metastore()`.
- A2 Aiven throughput is designed for 125/125 KiB/s.
- A3 You can run `sudo` in WSL for Docker.
- A4 Your laptop stays on while the bridge runs (Path B).
- A5 About 0.7 GB of raw synthetic data fits Free Edition's fair use. There is no published storage cap.
- A6 Free Edition's daily quota is unpublished, so we measure as we go.
- A7 The dev and prod pipelines read the same landing folder (`landing_root` variable).
- A8 The 4-week fixed horizon is acceptable for all three experiments.

Open questions (each resolved by a named probe, mostly in Phase 0)

ID	Question	Resolved by
O1	Does Path A work, with or without LinkedIn verification?	T1–T5
O2	Can we create catalogs (SQL), groups usable in UC grants, secret scopes, and service principals with OAuth secrets?	Phase 0 probes
O3	Does "one active pipeline per pipeline type" block *creating* a second pipeline, or only *running* concurrently?	Phase 0 probe
O4	Do Karapace's internal topics count against the 5-topic limit, and can the schema registry be disabled?	Phase 0
O5	How many Aiven users/ACLs are allowed? Is SASL off by default?	Phase 0
O6	Is PyPI reachable from serverless (statsmodels)?	Phase 0
O7	Does the `secret()` SQL function work inside pipeline expressions (PII pepper), and does any literal leak into the query profile or event log?	Phase 3
O8	Do column masks work on AUTO CDC SCD2 targets?	Phase 3
O9	Are the synthetic `99…` document and `+57 399…` phone ranges truly unassigned in Colombia? Else pick another obviously-fake scheme.	Phase 1
O10	Does the PEM truststore work with the shaded Kafka client on serverless (Path A only)?	T4

Appendix A. Repo tree (target)

```
D:\abtesting  (github.com/Pipe199x/fintech-ab-platform, public, MIT)
├── README.md  LICENSE  databricks.yml  pyproject.toml  uv.lock  .python-
version  .env.example  .gitignore  .pre-commit-config.yaml
├── .github/workflows/{ci.yml, deploy-dev.yml}
├── config/{topics.yml, simulation.yml, experiments.yml, gold_views.yml}
├── contracts/{envelope.v1|v2, payments/*, lending/*, savings/*, experiments/*, ops/*}.schema.json
├── src/fintech_ab/
│   ├── common/      config, assignment (hash), kafka config, time utils, logging
│   ├── producer/    population, behaviors, effects, faults, ratelimit, cli (backfill|live)
│   ├── bridge/      consumer loop, reconcile, writers (Files API | local dir)
│   ├── monitor/     guardrail windows, alerts, DLQ, lag
│   ├── transforms/  bronze, parse/route, explode, quality rules, pii, users, gold sql helpers
│   ├── gold/        YAML spec model, schema DDL builder, data dictionary
│   ├── stats/       ztest, welch, bootstrap, cuped, delta, multiplicity, srm, power
│   └── tools/       topics, groups, kafka_probe, volume_probe, sql runner, manifest
├── pipelines/fab/{bronze.py, silver_*.py, silver_users_scd2.py, gold_factory.py, ops_registry.py}
├── notebooks/{00_phase0_probes, 10_ab_analysis, 11_validation, 20_dq_report, 30_sss_payments,
40_delta_ops}.py
├── resources/{fab_pipeline, fab_daily_job, sss_payments_job, uc_schemas_volume, secret_scope,
dashboards}.yml
├── sql/{bootstrap/, governance/, verify/phase0..8.sql}
├── dashboards/{experimentation, business}.lvdash.json     powerbi/fintech_ab_exec.pbit
├── docker/compose.yml     perf/spark_ui_lab/{generate_data.py, run_lab.py}
├── tests/{unit, contract, integration, stats, pipeline}/
├── tools/{gen_data_dictionary.py, render_docs.sh}
└── docs/{plan/(PLAN.md, PLAN.pdf, diagrams/), adr/ADR-001..005, architecture/, prereg/, memos/, examples/,
          data_dictionary.md, runbook.md, evidence/phase0..8/, perf/}
```

Appendix B. Deliverables → phase

Deliverable	Where	Phase
0 PLAN.pdf	docs/plan	post-approval
1 Diagrams + ADR-001…005	docs/architecture, docs/adr	0 (002, 005), 2 (001), 3 (004), 6 (003), finalized in 8
2 Repo tree	Appendix A, materialized from Phase 0	0–1
3 Code, file by file	src/, pipelines/, notebooks/, resources/	1–7
4 Catalog of 41 views	§7, then generated from YAML into docs/data_dictionary.md	4
5 Runbook	docs/runbook.md, grown each phase	0–8
6 README	README.md	8

Verification (end-to-end)

At the end of Phase 6, one `databricks bundle run -t prod fab_daily` on prod data must chain the following, and `ops.validation_results` must be all-PASS:

1. The pipeline (Bronze → Silver → 41 Gold MVs), with reconciliation passing.
2. Governance.
3. The analysis: 3 decisions, each following its pre-registered rule.
4. Validation:
   - (a) truth inside the CIs;
   - (b) A/A FPR in [3.65%, 6.35%];
   - (c) SRM detected for EXP-SRM-900 only.
5. The DQ report.

CI must also be green, with unit, contract, stats (coverage + statsmodels oracle) and Kafka integration tests, including the bridge kill test. Phase 8's clean-clone runbook dry run is the final gate.